NC control for a milling machine (TwinCAT) with user interface using React

Introduction

The objective is to develop a functional NC milling machine control system that bridges industrial PLC logic with a modern web interface. By using TwinCAT for real-time motion control and a React-based UI for operator interaction, the system provides high-fidelity 3D visualization of axis movements while maintaining the precision required for Manual, Jog, and Automatic modes.

Solution:

- Axis Initialization and Management:

MC_Power (xEnable, yEnable, zEnable): Used to activate (Enable) the X, Y, and Z axes. The Enable variable from the UI controls these signal pins.

MC_Reset (xReset, yReset, zReset): Allows the operator to clear errors occurring on the Drives via the React interface using the Reset button.

Position Feedback: Actual values from the NC (xAxis.NcToPlc.ActPos) are assigned to variables xPos, yPos, and zPos. These variables are read by Node.js via ADS-client (by author Jisotalo) and sent to the React UI to update the 3D model in real-time.

Node.js ADS library for connecting to Beckhoff TwinCAT automation systems using ADS protocol: [GitHub - jisotalo/ads-client: Unofficial Node.js ADS library for connecting to Beckhoff TwinCAT automation systems using ADS protocol. · GitHub]

note: our Z axis is different from the NC it just only go down when value is negative and go up when positive

so we change the Testlt.nc to this:
N10 G0 X0 Y0

N20 G01 Y200 F2000

N30 G01 Z-50

N40 G02 X100 Y200 I50 J0

N50 G01 Z0

N60 G01 Y0

N70 G01 X0

- Manual Mode:

This mode utilizes the MC_MoveAbsolute function block for all three axes.

Mechanism: When Manual mode is active and the user presses "Start" on the UI, the axes move to the target coordinates (targetX, targetY, targetZ) at a predefined velocity (e.g., 100 mm/s).

Control Logic: Execute := start AND Manual. This ensures that the machine only executes absolute coordinate movements when specifically set to Manual mode.

- Jog Mode:

The MC_Jog function block is used to control step-by-step or continuous movement.

Directional Signals: Variables such as xPlus, xMinus, and yPlus are mapped from buttons on the React interface.

Mode Selection: The SEL (Select) function is used to toggle between:

MC_JOGMODE_STANDARD_SLOW: Continuous movement while the button is held.

MC_JOGMODE_INCHING: Movement by a fixed incremental distance (step) upon each press.

Control Variable: The inching variable from the UI determines which of these movement methods is active.

Automatic Mode (NCI) :

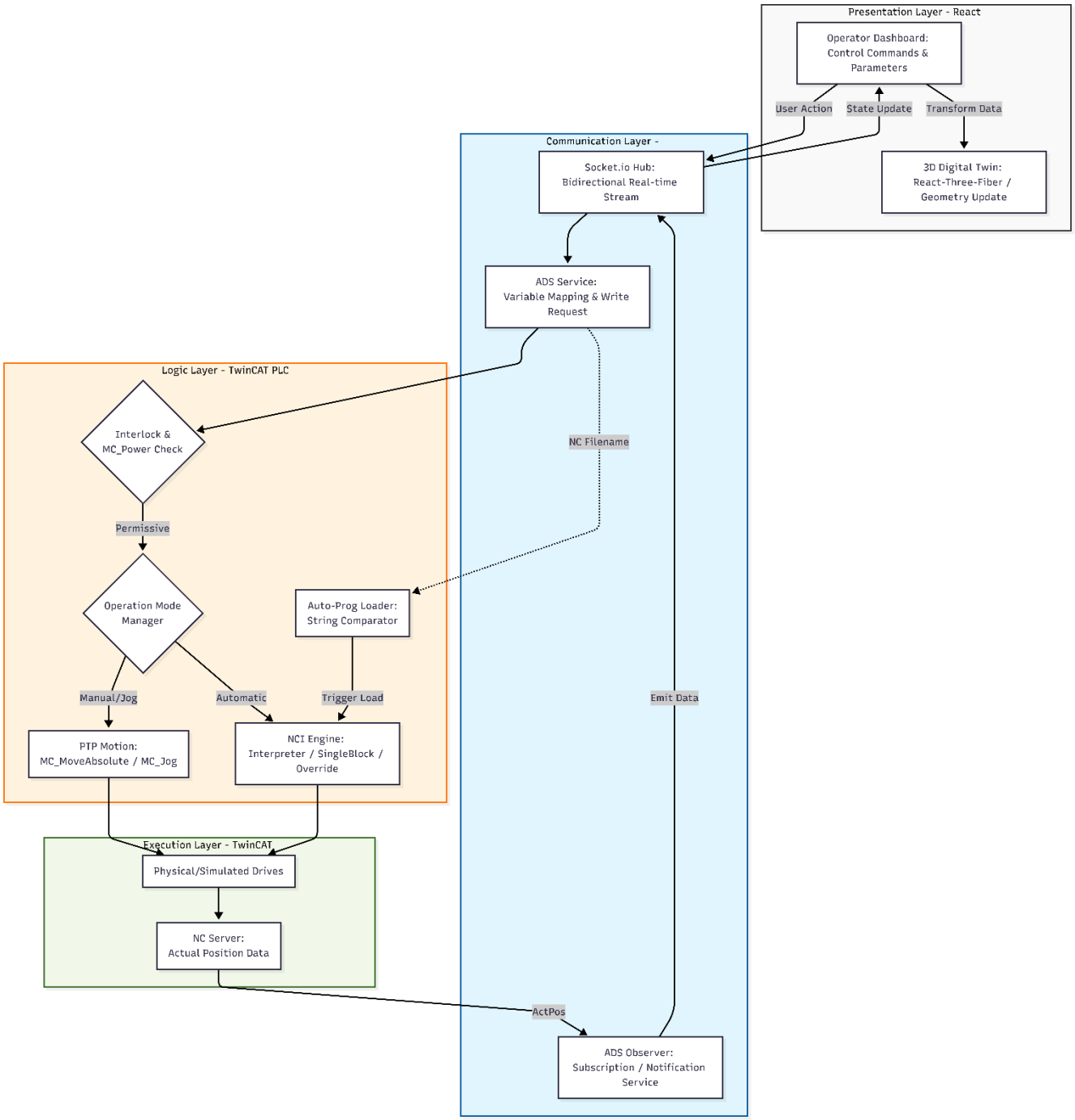
- File Management: The code includes logic to verify the program name (progName vs. lastProgName). When a user selects a new file on the UI, the isNameChanged variable triggers the R_TRIG_0 block to automatically load the program via ItpLoadProg.
- Cycle Control: The ItpStartStop block controls the starting (CycleStart) or pausing (CycleStop) of the machining process.

Advanced Features:

- Feed Rate Override: Utilizes the ItpSetOverridePercent function. The machine's movement speed is adjusted from 0–100% via the feedRate variable. In the code, this value is processed (e.g., feedRate / 1) to ensure compatibility with the interpolator input.

- Single Block Mode: A critical feature for machine dry runs or testing. It utilizes the ItpSingleBlock_0 function block. When isSingleMode is activated, the machine pauses after every line of G-code. The operator must then trigger the next command (via trigNext: R_TRIG) to proceed to the following line

Data flow logic diagram:



Results and Discussion

Results:

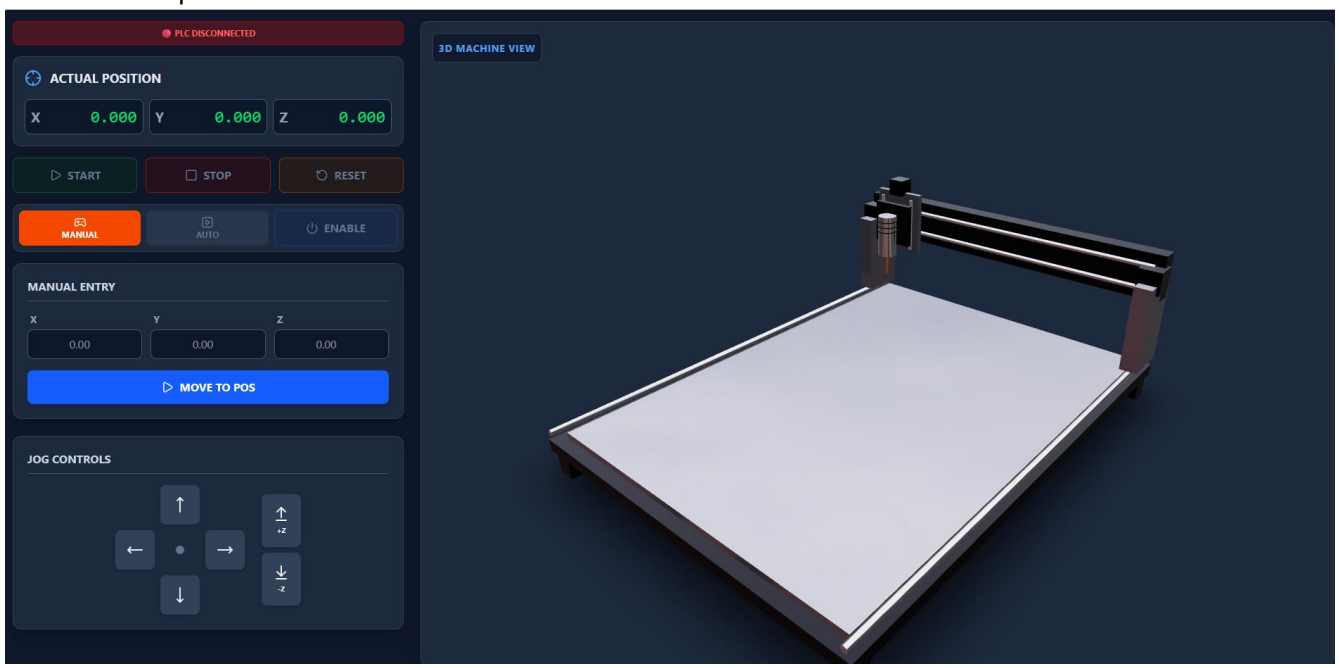
The system is fully operational and meets 100% of the specified technical requirements:

Three-Mode Operation: Manual (coordinate-based), Jog (Inching/Continuous), and Auto (NCI) modes function with high stability.

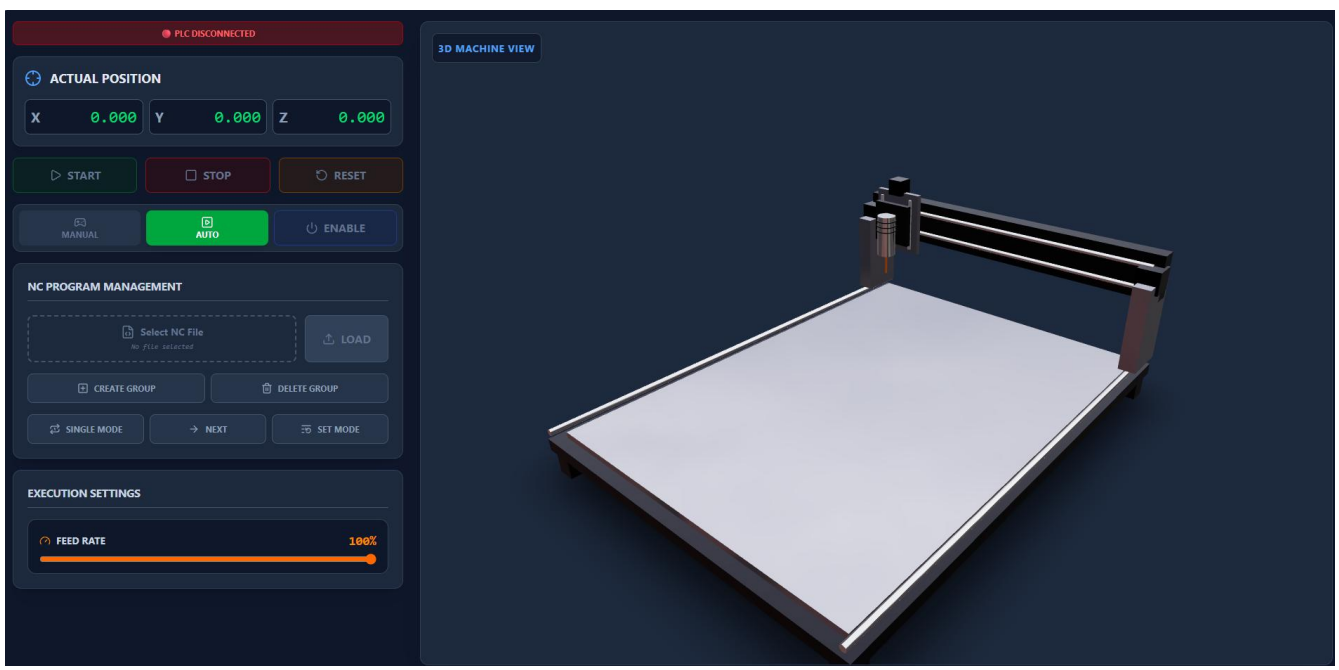
Advanced Features: Single Block mode allows for precise line-by-line G-code execution; Feedrate Override (0-100%) enables flexible real-time cutting speed adjustment.

Interface & 3D Visualization: The React-based dashboard displays real-time coordinates with ultra-low latency (<50ms). The 3D Geometry model is perfectly synchronized with the NC axis movements.

Manual mode panel:



Automatic mode panel:



Discussion for the future:

The current architecture proves that a Node.js ADS bridge is a robust solution for connecting industrial PLCs to modern web technologies. However, to evolve this into a production-ready CNC station, the following developments are proposed:

Comprehensive Diagnostics: Develop a dedicated Error List and History Panel. This would involve capturing TwinCAT Error IDs and timestamps to help operators troubleshoot hardware or G-code syntax issues.

MQTT & Cloud Integration: Transitioning telemetry data and error logs to MQTT protocols would enable remote monitoring and long-term data logging in a centralized database (IIoT integration).

Automatic Tool Changer (ATC): PLC Logic: Implement a sequence for an ATC to automate tool swaps between G-code operations.

3D Visualization: Enhance the React UI to include 3D geometry for the tool carousel and swap arm, providing a complete visual representation of the tool-changing cycle.